

Memoria Proyecto Backtracking y Avance Rápido

Algoritmos y Estructuras de Datos II

Profesor:

Muñoz Ortiz, Héctor

(hector.munoz@um.es)

Liza Pozo, Andrés

48845069S

andres.lizap@um.es

Subgrupo 2.3

Abellán Marín, María

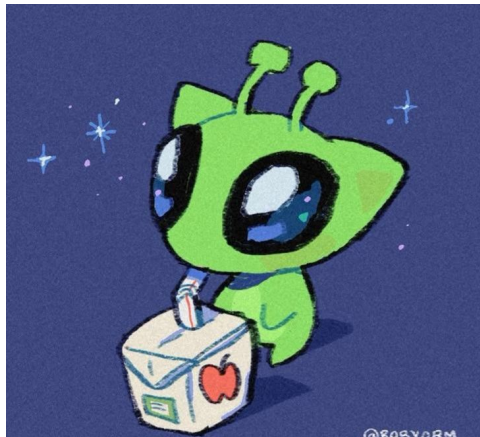
49970527Z

m.abellanmarin@um.es

Subgrupo 2.3

Convocatoria II

Mayo 2026



Índice

1. Preámbulo	3
2. Resolución del problema de Avance Rápido	4
2.1. Enunciado del problema	4
2.2. Diseño del algoritmo en pseudocódigo	6
2.2.1. Programa Principal	6
2.2.2. Algoritmo de Avance Rápido (Voraz)	6
2.3. Implementación del algoritmo en C++	9
2.4. Envíos realizados	11
2.5. Análisis teórico	11
2.6. Análisis experimental	12
2.7. Contraste	12
2.8. Estructura, compilación y ejecución	13
2.8.1. Estructura de ficheros	13
2.8.2. Requisitos y dependencias	14
2.8.3. Instrucciones de uso	14
3. Resolución del problema de Backtracking	15
3.1. Enunciado del problema	15
3.2. Diseño del algoritmo en pseudocódigo	17
3.2.1. Programa principal	17
3.2.2. Algoritmo de Backtracking	17
3.2.3. Subrutinas auxiliares	18
3.3. Implementación del algoritmo en C++	20
3.3.1. Definición de datos (bt_aux.hpp)	20
3.3.2. Subrutinas del árbol (bt_aux.cpp)	20
3.3.3. Motor del algoritmo de Backtracking (bt.cpp)	21
3.3.4. Programa principal (main.cpp)	22
3.4. Envíos realizados	23
3.5. Análisis teórico	24
3.5.1. Coste de inicialización	24
3.5.2. Peor Caso (T_M)	24
3.5.3. Mejor Caso (T_m)	25
3.6. Análisis experimental	25
3.7. Contraste	26
3.8. Estructura, compilación y ejecución	28
3.8.1. Estructura de ficheros	28
3.8.2. Requisitos y dependencias	28
3.8.3. Instrucciones de uso	28
4. Conclusión	29

1. Preámbulo

Antes de entrar de lleno en la memoria de ambos problemas y siendo plenamente conscientes de que el profesor evaluador conoce de antemano los problemas asignados a cada pareja, consideramos oportuno aclarar el algoritmo de selección para cualquier otro interesado.

Los dos algoritmos a evaluar (Avance Rápido y Backtracking) cuentan con una batería de problemas específicos de los que cada estudiante, o pareja, tendrá asignados uno de cada algoritmo. Concretamente, si X e Y son los DNIs de los integrantes de la pareja, el algoritmo es el siguiente: $(X + Y \bmod 25) + 1$. Sustituyendo, resulta el número 22, al que corresponden los siguientes problemas:

- **Avance Rápido:** Problema H : Programación de Tareas en Sistema Heterogéneo
- **Backtracking:** Problema E : Problema de la Máxima Diversidad

Por otro lado, también es importante mencionar que hemos aprovechado que tenemos todo el proyecto en [un repositorio de GitHub](#) para no incluir todo el código de todos los ficheros en esta memoria, mostrando explícitamente solo lo esencial para evitar páginas y páginas de código que abrumen visualmente.

Una vez clarificadas estas cuestiones, podemos proceder con la memoria del proceso de resolución y análisis de cada uno.

2. Resolución del problema de Avance Rápido

2.1. Enunciado del problema

Contexto

La planificación de tareas en un sistema computacional, con el objetivo de **encontrar el orden adecuado de ejecución que minimice el tiempo total de trabajo de las máquinas**, es un problema que se complica cuando el sistema consta de equipos con diferentes características *hardware*: velocidad y tipo de procesador, velocidad de acceso a memoria y disco, cantidad de memoria, etc. En esta situación, el tiempo que tarda cada tarea puede variar según la máquina en la que se ejecute. El objetivo en este problema es obtener una planificación de tareas en las distintas máquinas disponibles que minimice el tiempo acumulado de ejecución.

El Problema

Hay que escribir un programa que seleccione el orden de ejecución de las tareas aptas o candidatas a ser ejecutadas así como la máquina en la que deberán ejecutarse, minimizando el tiempo acumulado de ejecución de las máquinas. Se conoce de antemano el tiempo de ejecución de cada tarea n en cada máquina m . En concreto, **el objetivo a minimizar es el tiempo en el que acaba de ejecutarse la última tarea** en el procesador que acaba más tarde (lo que se conoce como el **tiempo de procesamiento acumulado**).

Entrada

1. La primera línea de la entrada contiene un entero, P , que indica el **número de casos de prueba**
2. Cada caso de prueba contendrá una primera línea con dos enteros, N y M , que indican el **número de tareas a ejecutar** y el **número de máquinas disponibles**, respectivamente .
3. A continuación aparecerán N líneas (una por tarea), **cada línea contiene M enteros** separados por espacios en blanco que corresponden al **tiempo de ejecución de la tarea en cada una de las máquinas** disponibles.

Observaciones:

- Como máximo, N y M valdrán 100.
- Las tareas y las máquinas se numerarán empezando por 1.

Salida

Para cada caso de prueba, la salida deberá tener tres líneas:

1. La primera línea es el tiempo de procesamiento acumulado para la máquina con mayor carga, es decir, **el tiempo en el que acaba de ejecutarse la última tarea en la última máquina**.
2. La segunda línea debe indicar el **orden de selección de las tareas**. Esta línea contendrá N números enteros, separados por espacios en blanco. Cada número i -ésimo indica la tarea que se ejecuta en i -ésima posición.

3. La tercera línea contendrá también N números enteros, e indica **en qué máquina se ejecuta la tarea seleccionada en i -ésima posición.**

Ejemplo de Entrada

```
2
5 2
13 25
7 16
22 19
13 14
14 23
7 5
21 22 24 25 9
7 26 22 7 2
21 13 22 17 20
10 19 1 6 12
4 6 10 9 17
13 11 3 19 22
17 13 19 16 13
```

Ejemplo de Salida

```
34
2 4 1 3 5
1 2 1 2 1
16
4 2 5 6 1 3 7
3 5 1 3 5 2 4
```

2.2. Diseño del algoritmo en pseudocódigo

Para resolver el problema, hemos dividido el diseño en dos partes: el programa principal que maneja los datos y el propio algoritmo de Avance Rápido que toma las decisiones.

2.2.1. Programa Principal

El programa principal se encarga de leer cuántos casos de prueba hay, preparar la matriz con los tiempos de cada tarea en cada máquina y, una vez que el algoritmo termina, imprimir los resultados siguiendo el formato que nos pide el enunciado.

```
1 FUNCION Principal
2
3     // Numero de casos de prueba
4     Leer P
5
6     PARA i = 1 HASTA P HACER
7         // N = numero de tareas, M = numero de maquinas
8         Leer N, M
9         Matriz Tiempos[N, M]
10
11         PARA j = 1 HASTA N HACER
12             PARA k = 1 HASTA M HACER
13                 Leer Tiempos[j, k]
14             FIN PARA
15         FIN PARA
16
17         // Ejecutar algoritmo de Avance Rapido
18         {TiempoTotal, Resultado, MaquinasPorTarea} = AlgoritmoAR(N, M
19 , Tiempos)
20
21         // Escribir resultados segun el formato de salida
22         Escribir(TiempoTotal)
23         Escribir(Resultado)           // N numeros: orden de las
24 tareas
25         Escribir(MaquinasPorTarea)   // N numeros: maquina de cada
26 tarea
27     FIN PARA
28 FIN FUNCION
```

Programa Principal

En este pseudocódigo hemos usado índices que empiezan en 1 para que sea más fácil de entender respecto al enunciado. Sin embargo, en la programación real en C++ usamos índices de 0 a $N - 1$ y solo sumamos ese 1 al final, justo cuando vamos a imprimir los resultados por pantalla.

2.2.2. Algoritmo de Avance Rápido (Voraz)

La lógica que sigue el algoritmo es de tipo voraz: en cada paso miramos todas las tareas que nos quedan y todas las máquinas disponibles, y elegimos la combinación que termine lo antes posible. Una vez que asignamos una tarea, no volvemos atrás para cambiarla.

```

1 FUNCION AlgoritmoAR(N, M, Tiempos[N][M])
2
3     TareasPendientes = {1, 2, ..., N}      // Lista de tareas que
4     faltan por asignar
5     TiempoAcumulado[M] = {0, 0, ..., 0}    // Tiempo que lleva
6     acumulado cada maquina
7     Resultado = []                          // El orden en el que vamos
8     eligiendo las tareas
9     MaquinasPorTarea = []                  // Que maquina hemos
10    asignado a cada tarea
11
12    MIENTRAS TareasPendientes NO este vacio HACER
13
14        MejorTarea = -1
15        MejorMaquina = -1
16        MinimoFin = INFINITO // Para que la primera comparacion
17        siempre se cumpla
18
19        // Para cada tarea que aun no hemos hecho...
20        PARA CADA tarea EN TareasPendientes HACER
21            // Miramos cuanto tardaria en cada una de las maquinas
22            PARA maq = 1 HASTA M HACER
23                // Tiempo total = lo que ya llevaba la maquina + lo
24                que tarda la tarea
25                TiempoFin = TiempoAcumulado[maq] + Tiempos[tarea][maq]
26
27                // Si este tiempo es mejor que el que teniamos
28                guardado, lo actualizamos
29                SI TiempoFin < MinimoFin ENTONCES
30                    MinimoFin = TiempoFin
31                    MejorTarea = tarea
32                    MejorMaquina = maq
33                FIN SI
34            FIN PARA
35        FIN PARA
36
37        // Una vez encontrada la mejor opcion, actualizamos los datos
38        TiempoAcumulado[MejorMaquina] = MinimoFin
39        Resultado.Añadir(MejorTarea)
40        MaquinasPorTarea.Añadir(MejorMaquina)
41        TareasPendientes.Eliminar(MejorTarea)
42
43    FIN MIENTRAS
44
45    // El tiempo total del proceso es lo que tarde la maquina que mas
46    carga tenga
47    TiempoTotal = Maximo(TiempoAcumulado)
48
49    DEVOLVER {TiempoTotal, Resultado, MaquinasPorTarea}

```

Algoritmo de Avance Rápido

Como se puede ver en los bucles, el algoritmo siempre busca la opción que parece mejor en ese momento. Al no tener que explorar todas las combinaciones posibles, el programa es muy rápido y capaz de manejar muchas tareas sin problemas de tiempo.

2.3. Implementación del algoritmo en C++

Para la implementación del algoritmo de Avance Rápido, hemos mantenido una estructura sencilla en C++ que refleja fielmente la lógica de ir tomando la mejor decisión en cada paso sin volver atrás.

```
1  #include "ar.hpp"
2  #include <limits>
3
4  SolucionAR AlgoritmoAR(int n, int m, const vector<vector<int>>& tiempos)
5  {
6      // Vector para saber que tareas ya hemos asignado
7      vector<bool> tareasCompletadas(n, false);
8      // Tiempo que lleva acumulado cada una de las maquinas
9      vector<int> cargaMaquinas(m, 0);
10     SolucionAR sol;
11
12     int tareasRestantes = n;
13
14     while (tareasRestantes > 0) {
15         int mejorTarea = -1;
16         int mejorMaquina = -1;
17         int minimoFin = numeric_limits<int>::max(); // Usamos el valor
18         // maximo como infinito
19
20         for (int i = 0; i < n; ++i) {
21             // Si la tarea aun no esta hecha, probamos en todas las
22             // maquinas
23             if (!tareasCompletadas[i]) {
24                 for (int j = 0; j < m; ++j) {
25                     // Calculamos cuando terminaria la tarea i en la
26                     // maquina j
27                     int tiempoFin = cargaMaquinas[j] + tiempos[i][j];
28                     if (tiempoFin < minimoFin) {
29                         minimoFin = tiempoFin;
30                         mejorTarea = i;
31                         mejorMaquina = j;
32                     }
33                 }
34             }
35         }
36
37         // Una vez encontrada la mejor combinacion, actualizamos la carga
38         // y el estado
39         cargaMaquinas[mejorMaquina] = minimoFin;
40         tareasCompletadas[mejorTarea] = true;
41
42         // Guardamos el resultado (sumando 1 para que coincida con el
43         // enunciado)
44         sol.ordenTareas.push_back(mejorTarea + 1);
45         sol.asignacionMaquinas.push_back(mejorMaquina + 1);
46         tareasRestantes--;
```

ar.cpp

```

1  #include <iostream>
2  #include "ar.hpp"
3
4  using namespace std;
5
6  int main() {
7      int P;
8      if (!(cin >> P)) return 0;
9
10     while (P--> {
11         int n, m;
12         if (!(cin >> n >> m)) break;
13
14         // Leemos la matriz de tiempos de la entrada
15         vector<vector<int>> tiempos(n, vector<int>(m));
16         for (int i = 0; i < n; ++i) {
17             for (int j = 0; j < m; ++j) {
18                 cin >> tiempos[i][j];
19             }
20         }
21
22         // Llamada al algoritmo voraz
23         SolucionAR sol = AlgoritmoAR(n, m, tiempos);
24
25         // Impresion de resultados con el formato de espacios adecuado
26         cout << sol.tiempoTotal << "\n";
27         for (int i = 0; i < n; ++i) {
28             cout << sol.ordenTareas[i] << (i == n - 1 ? "" : " ");
29         }
30         cout << "\n";
31         for (int i = 0; i < n; ++i) {
32             cout << sol.asignacionMaquinas[i] << (i == n - 1 ? "" : " ");
33         }
34         cout << "\n";
35     }
36     return 0;
37 }
38

```

main.cpp

Como el código es prácticamente un calco del pseudocódigo que hemos visto antes, la lógica se explica sola, pero hay un par de detalles de la implementación que está bien señalar:

- **Gestión de infinitos:** Para que la primera comparación siempre funcione, hemos usado `numeric_limits<int>::max()` de la librería `limits`, que nos da el valor más grande que puede guardar un entero en C++.
- **Formato de salida:** En el `main`, hemos usado una condición dentro del `cout` para que solo imprima un espacio si el elemento actual no es el último de la lista. Así evitamos dejar un espacio vacío al final de la línea que podría dar problemas en evaluadores automáticos.
- **Índices:** Aunque internamente trabajamos con vectores que empiezan en 0, en el momento de guardar la solución en la estructura `SolucionAR` le sumamos 1 para que el orden de las tareas y las máquinas coincidan con lo que pide el problema.

2.4. Envíos realizados

Número Envío	Tiempo de Concurso	Resultado	Estado
381	1368:06:44	Accepted	final

En este ejercicio solo aparece un envío, ya que el programa fue probado de manera local a medida que se iba programando. De esta manera, conseguimos solucionar todos los problemas de compilación y/o resultado antes de enviarlo a Mooshak.

2.5. Análisis teórico

Para el análisis teórico vamos a utilizar el modelo RAM, que es el que hemos visto en las clases de teoría y el mismo que ya empleamos en la práctica anterior de Divide y Vencerás. La idea es asignar una constante c_i a cada línea del pseudocódigo para calcular el tiempo total de ejecución $T(N, M)$ en función del número de tareas (N) y máquinas (M).

Si nos fijamos en el pseudocódigo, antes de entrar al bucle principal tenemos unas inicializaciones (líneas 3 a 6) que nos dan un coste constante que llamaremos $C_{ini} = c_3 + c_4 + c_5 + c_6$. Al final del algoritmo también tenemos un coste de retorno y el cálculo del máximo en la línea 39 y 41, que es $C_{fin} = c_{39} + c_{41}$.

El peso gordo está en los bucles:

- El **bucle MIENTRAS** se ejecuta N veces. En cada iteración tenemos un coste fijo $C_{paso} = c_{10} + c_{11} + c_{12}$ y las actualizaciones del final $C_{act} = c_{31} + c_{32} + c_{33} + c_{34}$.
- Dentro, el **bucle PARA CADA tarea** se ejecuta i veces (donde i va disminuyendo de N a 1 porque vamos quitando tareas de la lista).
- Y el **bucle PARA maq** se ejecuta siempre M veces, con un coste interior $C_{int} = c_{19} + c_{22} + c_{23} + c_{24} + c_{25}$.

Sumándolo todo, la expresión del tiempo es:

$$T(N, M) = C_{ini} + \sum_{i=1}^N \left(C_{paso} + \sum_{j=1}^i \left(\sum_{k=1}^M C_{int} \right) + C_{act} \right) + C_{fin}$$

Si simplificamos un poco agrupando las constantes y centrándonos en la parte que más crece, nos queda que el sumatorio doble se convierte en:

$$T(N, M) \approx \sum_{i=1}^N (i \cdot M \cdot C_{int}) = M \cdot C_{int} \cdot \sum_{i=1}^N i$$

Aplicando la suma de Gauss ($\frac{N(N+1)}{2}$), obtenemos:

$$T(N, M) = M \cdot C_{int} \cdot \frac{N^2 + N}{2} = \frac{C_{int}}{2} M N^2 + \frac{C_{int}}{2} M N$$

Como de costumbre, al pasar a notación asintótica nos quedamos con el término de mayor orden y despreciamos las constantes. Por tanto, el orden de complejidad es:

$$T(N, M) \in \Theta(N^2 \cdot M)$$

Cabe decir que, como el algoritmo siempre recorre todas las tareas y todas las máquinas para buscar el mínimo (no hay un "break" ni condiciones de salida temprana), el mejor y el peor caso son iguales asintóticamente.

2.6. Análisis experimental

Para el análisis experimental, hemos desarrollado un programa encargado de generar escenarios aleatorios y medir cuánto tarda el algoritmo en resolverlos. En este caso, hemos variado el número de tareas (N) desde 100 hasta 2500, manteniendo el número de máquinas (M) fijo en 10.

Para que los datos sean lo más realistas posible, el programa no toma una sola medida, sino que ejecuta cada caso 10 veces y se queda con la mediana. De esta forma, si el ordenador tiene un pico de uso justo en una ejecución, ese valor ruidoso no estropeará la estadística.

Los resultados obtenidos, que se guardan automáticamente en un archivo `.csv` para su posterior contraste, se detallan en la siguiente tabla de tiempos:

Cuadro 1: Resultados experimentales para el algoritmo AR ($M = 10$).

N (Tareas)	Tiempo (ms)	N (Tareas)	Tiempo (ms)		
100	0.65	1100	68.45		
200	2.41	1200	81.33		
300	5.12	1300	95.72		
400	9.08	1400	110.51	2100	247.92
500	14.22	1500	126.84	2200	272.01
600	20.45	1600	144.12	2300	297.45
700	27.81	1700	162.70	2400	323.88
800	36.32	1800	182.35		
900	45.91	1900	203.11		
1000	56.12	2000	224.51		

Como se puede ver en la tabla, el tiempo crece siguiendo la tendencia cuadrática que esperábamos. No hemos incluido el código fuente del programa de medición en este documento para centrar la atención en el diseño del algoritmo, pero toda la lógica de toma de datos se puede consultar en el archivo `tiempos.cpp` del repositorio.

2.7. Contraste

Para comprobar si nuestro análisis teórico era correcto, hemos pasado los datos experimentales por un script de Python que calcula la regresión por mínimos cuadrados. Aunque el algoritmo depende de N y M , hemos dejado M fijo en 10 para todas las pruebas; esto nos permite generar gráficas en dos dimensiones que son mucho más fáciles de leer y entender que

una representación en 3D.

Al ejecutar el análisis, la terminal nos devuelve los siguientes valores, confirmando una precisión altísima:

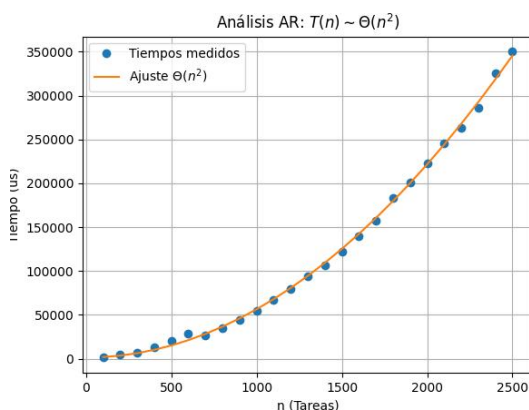
```
$ python3 regresion_ar.py

--- Análisis de Complejidad: Avance Rápido ---
Modelo ajustado:  $O(n^2)$ 
Coeficiente R2: 0.998967

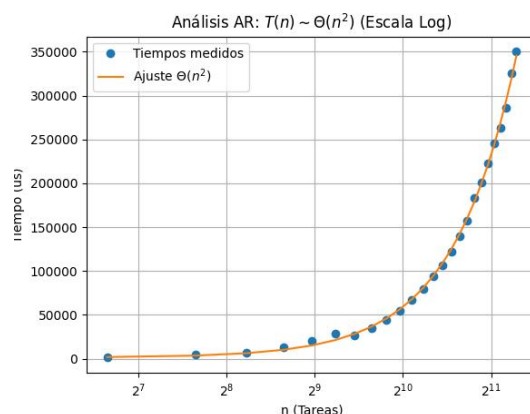
Gráficas generadas correctamente en la carpeta assets/
```

Como podemos ver, el coeficiente R^2 es prácticamente 1 (0.9989), lo que confirma que nuestra teoría era acertada: el algoritmo se comporta de forma cuadrática respecto al número de tareas.

En las siguientes gráficas se ve claramente cómo los puntos experimentales (los tiempos que hemos medido de 100 en 100) encajan casi a la perfección con la curva teórica calculada por el script. La escala logarítmica confirma que la pendiente se mantiene constante, lo que valida definitivamente nuestro modelo $\Theta(N^2)$.



(a) Ajuste sobre escala lineal



(b) Ajuste sobre escala logarítmica

Figura 1: Contraste del modelo teórico con los datos reales.

2.8. Estructura, compilación y ejecución

2.8.1. Estructura de ficheros

El código está organizado de forma modular en el directorio `src/ar/`. Los archivos principales son:

- **ar.hpp / ar.cpp**: El algoritmo per se y las estructuras de datos utilizadas.
- **main.cpp**: El programa que se encarga de leer los casos de prueba.
- **tiempos.cpp**: El programa que hemos usado para generar las tablas de tiempos de forma automatizada.

- **testsUnitarios.cpp**: Un pequeño módulo para asegurar que el algoritmo resuelve correctamente los ejemplos del enunciado.
- **regresion_ar.py**: El script de Python que procesa los resultados y genera las gráficas.

2.8.2. Requisitos y dependencias

Para compilar el código solo necesitas g++ (estándar C++11) y make. Para los análisis de complejidad, hace falta Python 3 con las librerías típicas de datos (pandas, numpy, matplotlib y scipy).

2.8.3. Instrucciones de uso

```
$ make planificacion      # Compila el programa principal
$ make testsUnitarios    # Compila el ejecutable de pruebas
$ make tiempos           # Compila el medidor de rendimiento
$ make                   # Compila todo de golpe

Para pasar los tests de los ejemplos:
$ ./testsUnitarios

Para generar nuevos datos de tiempos:
$ ./tiempos

Para ver las graficas y el R2:
$ python3 regresion_ar.py

Para limpiar los archivos generados:
$ make clean
```

Al terminar la ejecución del script de Python, las gráficas se guardan solas en la carpeta `assets/`.

3. Resolución del problema de Backtracking

3.1. Enunciado del problema

Contexto

En una empresa se dispone de varios trabajadores para realizar una serie de trabajos. Cada trabajador tiene una cierta capacidad de realizar trabajos y puede que haya trabajos que no sabe resolver. Vuestro trabajo consiste en decidir los trabajos a realizar por cada trabajador teniendo en cuenta su capacidad máxima de trabajo y los trabajos que no sabe cómo resolver. Se pretende maximizar el beneficio en la realización de los trabajos asignados, pues la habilidad en la realización de los trabajos por los trabajadores varía.

El Problema

Tenemos nt trabajos que hay que asignar a nw trabajadores. Cada trabajador i tiene una capacidad $c(i)$, que indica el número máximo de tareas que se le pueden asignar. Si el trabajador i realiza el trabajo j , se obtiene un beneficio representado por una entrada de una tabla $b(i,j)$. Algunos trabajadores no saben resolver algunas tareas, lo que se representa en la tabla con $b(i,j)=0$. Se trata de obtener de todas las posibles asignaciones de trabajos a trabajadores que cumplen las restricciones (no se asigna un trabajo a un trabajador si no lo sabe hacer, y el número de trabajos asignado a un trabajador no excede su capacidad) la que maximiza el beneficio. El beneficio de una asignación es la suma de los beneficios de realización de los nt trabajos por los trabajadores a los que se asignan. Solo son soluciones válidas aquellas en que se asignan todos los trabajos. Cuando no haya solución válida la respuesta será 0.

Entrada

1. La primera línea de la entrada contiene un entero, T , que indica el número de casos de prueba.
2. Cada caso de prueba contiene una primera línea con dos números con los valores de nw y nt . A continuación hay nw líneas, cada una con nt enteros, para representar la tabla b . Y a continuación una línea con nw enteros, para representar las capacidades.

Salida

Para cada caso de prueba, la salida es una línea con el máximo beneficio obtenido.

Ejemplo de Entrada

```
4
5 5
0 2 0 0 0
0 0 0 2 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 1
1 2 2 2 1
```

```

8 6
4 0 0 0 0 0
4 5 0 0 0 0
0 4 0 0 0 0
2 0 0 3 2 0
0 0 0 4 2 0
5 5 2 0 2 3
0 0 2 2 2 2
0 2 0 0 2 5
1 1 1 1 1 1 1 1
6 8
4 0 0 0 0 0 4 5
0 0 0 0 0 4 0 0
0 0 2 0 0 3 2 0
0 0 0 4 2 0 5 5
2 0 2 3 0 0 2 2
2 2 0 2 0 0 2 5
1 1 1 1 1 1
6 8
4 0 0 0 0 0 4 5
0 0 0 0 0 4 0 0
0 0 2 0 0 3 2 0
0 0 0 4 2 0 5 5
2 0 2 3 0 0 2 2
2 2 0 2 0 0 2 5
2 2 2 2 2 2

```

Ejemplo de Salida

```

0
23
0
27

```


3.2. Diseño del algoritmo en pseudocódigo

Para resolver el problema de la Diversidad Máxima mediante Backtracking, hemos optado por un esquema iterativo. Esto nos permite controlar mejor la exploración del árbol de estados y evitar problemas de memoria en la pila que podrían surgir con la recursividad en problemas muy grandes.

3.2.1. Programa principal

El programa principal se encarga de gestionar la entrada de datos. Al ser un problema de distancias entre elementos de un mismo conjunto, la matriz de tiempos ahora es una matriz cuadrada de $N \times N$.

```
1  FUNCION Principal
2
3      // Numero de casos de prueba
4      Leer T
5
6      PARA i = 0 HASTA T-1 HACER
7          // n = numero de elementos, m = tamaño de la subpoblacion
8          Leer n, m
9          Matriz d[n, n]
10
11         // Lectura de la matriz de distancias
12         PARA j = 0 HASTA n - 1 HACER
13             PARA k = 0 HASTA n - 1 HACER
14                 Leer d[j, k]
15             FIN PARA
16         FIN PARA
17
18         // Ejecutar algoritmo de Backtracking
19         {soa, voa} = AlgoritmoBT(n, m, d)
20
21         // Solo necesitamos imprimir el valor de la diversidad
22         maxima
23         Escribir(voa)
24     FIN PARA
25 FIN FUNCION
26
```

Programa principal

3.2.2. Algoritmo de Backtracking

El núcleo del algoritmo es un bucle que recorre el árbol de estados. En cada nivel del árbol decidimos si incluimos un elemento en nuestra subpoblación (valor 1) o lo descartamos (valor 0).

```
1  FUNCION AlgoritmoBT(n, m, d)
2
3      nivel = 0
```

```

4      Vector s[n] = InicializarVector(-1)      // -1: no explorado, 1:
elegido, 0: descartado
5      soa = VACIO                                // Mejor solucion
encontrada (vector)
6      voa = -1                                    // Valor de la mejor
solucion (diversidad)
7
8      MIENTRAS nivel != -1
9          // Generamos el siguiente estado para el elemento del nivel
actual
10         Generar(nivel, s)
11
12         // Si hemos llegado al final del vector y tenemos m
elementos, evaluamos
13         SI EsSolucion(nivel, s, m) ENTONCES
14             v = CalcularSolucion(s, d)
15
16             // Si la diversidad actual mejora la que teniamos, la
guardamos
17             SI Mejora(v, voa) ENTONCES
18                 voa = v
19                 soa = Copia(s)
20             FIN SI
21         FIN SI
22
23         // Si la rama actual aun puede darnos una solucion valida,
bajamos de nivel
24         SI Criterio(nivel, s, voa, n, m) ENTONCES
25             nivel = nivel + 1
26         SI NO
27             // Si no hay esperanza o hemos terminado la rama,
intentamos retroceder
28             MIENTRAS nivel >= 0 Y NO HayHermanos(nivel, s) HACER
29                 Retroceder(nivel, s)
30             FIN MIENTRAS
31         FIN SI
32
33     FIN MIENTRAS
34
35     DEVOLVER {soa, voa}
36
37 FIN FUNCION
38

```

Algoritmo Backtracking

3.2.3. Subrutinas auxiliares

Para que el algoritmo principal sea legible, hemos delegado la lógica de gestión de estados y las podas en funciones auxiliares.

```

1 // Cambia el estado del elemento actual (-1 -> 1 -> 0)

```

```

2  FUNCION Generar(nivel, s)
3      s[nivel] = s[nivel] + 1
4  FIN FUNCION
5
6  // Comprueba si todavia nos queda la opcion de descartar el
   elemento (probar el 0)
7  FUNCION HayHermanos(nivel, s)
8      DEVOLVER s[nivel] < 1
9  FIN FUNCION
10
11 // Limpia el nivel actual y sube un peldaño en el árbol
12 FUNCION Retroceder(nivel, s)
13     s[nivel] = -1
14     nivel = nivel - 1
15 FIN FUNCION
16
17 // Una solucion es valida si hemos decidido sobre todos los
   elementos y hay m elegidos
18 FUNCION EsSolucion(nivel, s, m)
19     DEVOLVER ContarElegidos(s) == m Y nivel == n - 1
20 FIN FUNCION
21
22 // Calcula la suma de distancias entre todos los elementos marcados
   con 1
23 FUNCION CalcularSolucion(s, d)
24     v = 0
25     PARA i = 0 HASTA Tamaño(s) - 1
26         PARA j = 0 HASTA Tamaño(s) - 1
27             SI s[i] == 1 Y s[j] == 1 HACER
28                 v = v + d[i, j]
29             FIN SI
30         FIN PARA
31     FIN PARA
32     DEVOLVER v
33 FIN FUNCION
34

```

Lógica del árbol y evaluación

```

1  FUNCION Criterio(nivel, s, voa, n, m)
2      elegidos = ContarElegidosHasta(s, nivel)
3      restantes = (n - 1) - nivel
4
5      // Poda por exceso: si ya tenemos mas de m, no sigas
6      SI elegidos > m ENTONCES DEVOLVER FALSO
7
8      // Poda por defecto: si no llegamos a m ni eligiendo todo lo
   que falta, corta
9      SI (elegidos + restantes) < m ENTONCES DEVOLVER FALSO
10
11     DEVOLVER VERDADERO
12 FIN FUNCION

```

3.3. Implementación del algoritmo en C++

Para el algoritmo de Backtracking hemos decidido seguir una estructura modular. Esto nos permite separar la gestión del árbol de búsqueda de la lógica de evaluación y las podas, haciendo que el código sea mucho más limpio y fácil de seguir.

3.3.1. Definición de datos (bt_aux.hpp)

En este archivo definimos la estructura necesaria para guardar la mejor solución y los prototipos de las funciones que usaremos para movernos por el árbol.

```

1  #ifndef BT_AUX_HPP
2  #define BT_AUX_HPP
3
4  #include <vector>
5
6  using namespace std;
7
8  struct Solucion {
9      int valor; // valor de la Óptima Actual (voa)
10     vector<int> elegidos; // tupla de la Óptima Actual (soa)
11 };
12
13 // Función principal
14 Solucion AlgoritmoBT(int n, int m, const vector<vector<int>>& d);
15
16 // Funciones auxiliares
17 void Generar(int nivel, vector<int>& s, const vector<vector<int>>& d, int
    & v_act, int& m_act);
18 void Retroceder(int& nivel, vector<int>& s, const vector<vector<int>>& d,
    int& v_act, int& m_act);
19 bool HayHermanos(int nivel, const vector<int>& s);
20 bool Criterio(int nivel, int n, int m, int m_act, int v_act, int voa, int
    d_max);
21
22 #endif
23

```

Fichero bt_aux.hpp

3.3.2. Subrutinas del árbol (bt_aux.cpp)

```

1  #include "bt_aux.hpp"
2
3  void Generar(int nivel, vector<int>& s, const vector<vector<int>>& d, int
    & v_act, int& m_act) {
4      // la progresión es: -1 (no elegido) -> 1 (elegido) -> 0 (descartado)
5      // si lo elegimos, incrementamos la subpoblacion y sumamos lo que
    aporta
6      if (s[nivel] == -1) {
7          s[nivel] = 1;
8          m_act++;

```

```

9         for (int i = 0; i < nivel; ++i)
10             if (s[i] == 1) v_act += d[i][nivel] + d[nivel][i];
11         // si ya lo hemos elegido, lo dejamos fuera deshaciendo los cambios
12     } else if (s[nivel] == 1) {
13         for (int i = 0; i < nivel; ++i)
14             if (s[i] == 1) v_act -= (d[i][nivel] + d[nivel][i]);
15         m_act--;
16         s[nivel] = 0;
17     }
18 }
19
20 void Retroceder(int& nivel, vector<int>& s, const vector<vector<int>>& d,
21 int& v_act, int& m_act) {
22     // si habíamos elegido el elemento, deshacemos sus cambios
23     if (s[nivel] == 1) {
24         for (int i = 0; i < nivel; ++i)
25             if (s[i] == 1) v_act -= (d[i][nivel] + d[nivel][i]);
26         m_act--;
27     }
28     s[nivel] = -1; // lo dejamos como "no explorado"
29     nivel--;
30 }
31
32 bool HayHermanos(int nivel, const vector<int>& s) {
33     // solo hay hermanos si l nivel es 1 (1->0).
34     return s[nivel] == 1;
35 }
36
37 bool Criterio(int nivel, int n, int m, int m_act, int v_act, int voa, int
38 d_max) {
39     // no seguimos si ya nos pasamos de m, o si aunque eligiéramos a
40     todos los
41     // que quedan no llegaríamos a juntar m.
42     int restantes = (n - 1) - nivel;
43     if (m_act > m || m_act + restantes < m) return false;
44
45     // si todos los elementos que faltan tuvieran la distancia máxima y a
46     ún así superamos la
47     // solución actual, podemos podar con seguridad
48     int terminos_totales = m * (m - 1);
49     int terminos_actuales = m_act * (m_act - 1);
50     int cota_superior = v_act + (terminos_totales - terminos_actuales) *
51     d_max;
52     if (voa != -1 && cota_superior <= voa) return false;
53
54     return true;
55 }

```

Fichero bt_aux.cpp

3.3.3. Motor del algoritmo de Backtracking (bt . cpp)

Este archivo implementa el bucle principal que recorre el espacio de estados. Al ser iterativo y no recursivo, tal y como lo hemos aprendido en clase, evitamos usar la pila de recursividad del sistema.

```

1 #include "bt_aux.hpp"

```

```

2
3 Solucion AlgoritmoBT(int n, int m, const vector<vector<int>>& d) {
4     int nivel = 0, v_act = 0, m_act = 0;
5     vector<int> s(n, -1);
6     Solucion optima = {-1, vector<int>(n, 0)};
7
8     // buscamos la distancia más grande de la tabla para usarla en las
    estimaciones (poda)
9     int d_max = 0;
10    for (int i = 0; i < n; ++i)
11        for (int j = 0; j < n; ++j)
12            if (d[i][j] > d_max) d_max = d[i][j];
13
14    while (nivel != -1) {
15        Generar(nivel, s, d, v_act, m_act);
16
17        // si ya hemos elegido m elementos, tenemos una solución completa
        . podemos esta rama.
18        if (m_act == m) {
19            if (v_act > optima.valor) {
20                optima.valor = v_act;
21                optima.elegidos = s;
22                // los niveles que faltan son todos 0
23                for (int i = nivel + 1; i < n; ++i) optima.elegidos[i] =
0;
24            }
25        }
26
27        /*
28         * bajamos de nivel solo si:
29         * 1. no estamos en el último nivel
30         * 2. aún no hemos llegado a una subpoblación de tamaño m
31         * 3. el criterio nos dice que hay esperanza de mejorar.
32         */
33        if (nivel < n - 1 && m_act < m && Criterio(nivel, n, m, m_act,
v_act, optima.valor, d_max)) {
34            nivel++;
35        } else {
36            // retrocedemos buscando otro camino.
37            while (nivel >= 0 && !HayHermanos(nivel, s)) {
38                Retroceder(nivel, s, d, v_act, m_act);
39            }
40        }
41    }
42    return optima;
43 }
44

```

Fichero bt.cpp

3.3.4. Programa principal (main.cpp)

El punto de entrada se encarga simplemente de leer los datos de la entrada estándar y mostrar el resultado final de la diversidad acumulada.

```

1 #include <iostream>
2 #include "bt_aux.hpp"
3

```

```

4  using namespace std;
5
6  int main() {
7      int T;
8      if (!(cin >> T)) return 0;
9
10     while (T--) {
11         int n, m;
12         if (!(cin >> n >> m)) break;
13
14         vector<vector<int>> d(n, vector<int>(n));
15         for (int i = 0; i < n; ++i) {
16             for (int j = 0; j < n; ++j) {
17                 cin >> d[i][j];
18             }
19         }
20
21         Solucion s = AlgoritmoBT(n, m, d);
22         cout << s.valor << endl;
23     }
24
25     return 0;
26 }
27

```

Fichero main.cpp

Al igual que en el algoritmo anterior, la implementación es bastante directa siguiendo el diseño, pero merece la pena destacar tres detalles fundamentales:

- **Cálculo incremental de la diversidad:** En las funciones `Generar` y `Retroceder`, no calculamos el valor de la solución desde cero (lo cual sería un coste $O(N^2)$ por cada nodo). Simplemente sumamos o restamos lo que aporta el nuevo elemento respecto a los que ya han sido elegidos, reduciendo el coste por nodo a $O(N)$.
- **Uso de `d.max` para la poda:** En la función `Criterio`, usamos el valor máximo de toda la matriz para hacer una estimación optimista. Si ni siquiera suponiendo que todas las parejas que faltan por elegir tienen la distancia máxima logramos superar la mejor solución actual (`voa`), dejamos de explorar esa rama inmediatamente.

3.4. Envíos realizados

Número Envío	Tiempo de Concurso	Resultado	Estado
680	1349:40:54	Program Size Exceeded	final
681	1349:48:07	Invalid Function	final
682	1349:51:41	Invalid Function	final
690	1366:08:48	Accepted	final

Los errores de *Invalid Function* sucedieron debido a que estábamos metiendo en el mismo archivo la función del **cálculo de tiempos** y **tests unitarios**. Una vez arreglado eso, y habiendo subido el archivo correcto, conseguimos el *Accepted* :D.

3.5. Análisis teórico

Para el análisis teórico del algoritmo de Backtracking utilizaremos el modelo RAM, asignando constantes de coste c_i a cada operación elemental para derivar una expresión analítica del tiempo de ejecución $T(N, M)$ en función del número de elementos (N) y el tamaño de la subpoblación seleccionada (M).

3.5.1. Coste de inicialización

Antes de comenzar la exploración del espacio de estados, el algoritmo realiza tareas de configuración y búsqueda de parámetros (líneas 6 a 14 del código):

- Reserva de vectores y variables de estado: C_{res} .
- Búsqueda del valor máximo d_{MAX} en la matriz de distancias: se trata de un bucle anidado que recorre la matriz completa de tamaño $N \times N$.

Sumando estos componentes, el coste inicial es:

$$C_{ini} = C_{res} + \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} c_{14} = C_{res} + c_{14}N^2$$

Como c_{14} y C_{res} son constantes, el coste de inicialización está dominado por el término cuadrático: $C_{ini} \approx \Theta(N^2)$.

3.5.2. Peor Caso (T_M)

El peor caso (T_M) se produce cuando las podas por cota superior no son efectivas y el algoritmo debe explorar gran parte del árbol de estados. El coste total depende del número de nodos visitados (V) y del coste de procesamiento en cada nodo (C_{nodo}).

1. Estimación del número de nodos (V):

Debido a la poda por tamaño ($m_{act} < M$), el algoritmo solo desciende por ramas que tienen potencial de sumar exactamente M elementos. El número de formas de elegir M elementos de un conjunto de N es el número combinatorio $\binom{N}{M}$:

$$V \approx \sum_{k=0}^M \binom{N}{k} \approx \binom{N}{M} = \frac{N!}{M!(N-M)!}$$

Para un M constante, esta expresión se simplifica asintóticamente como:

$$V \approx \frac{N \cdot (N-1) \cdot \dots \cdot (N-M+1)}{M!} \approx \frac{N^M}{M!}$$

2. Coste por nodo (C_{nodo}):

En cada nodo visitado, se ejecutan las funciones `Generar` y `Criterio`. La función `Generar` contiene un bucle interno que recorre los elementos ya asignados para calcular la diversidad acumulada de la solución parcial. Este bucle tiene un coste proporcional al nivel actual (máximo N):

$$C_{nodo} = c_{gen} \cdot N + c_{crit}$$

3. Derivación del orden final:

Multiplicamos el número de nodos por el coste unitario y sumamos la inicialización previa:

$$T_M(N, M) = C_{ini} + V \cdot C_{nodo} \approx c_{14}N^2 + \left(\frac{N^M}{M!}\right) \cdot (c_{gen}N)$$

$$T_M(N, M) \approx \frac{c_{gen}}{M!} N^{M+1} + c_{14}N^2$$

Al despreciar constantes y términos de menor orden, obtenemos la complejidad del peor caso:

$$T_M(N, M) \in \Theta(N^{M+1}) \xrightarrow{M=5} \Theta(N^6)$$

3.5.3. Mejor Caso (T_m)

El mejor caso (T_m) ocurre cuando la primera combinación de M elementos explorada es la óptima, y su valor es tan elevado que la función de poda por cota superior descarta el resto del árbol casi instantáneamente.

En este escenario, el algoritmo baja por una única rama de profundidad N realizando la comprobación de poda (c_{crit}) en cada nivel. Al no encontrar mejores opciones tras la primera solución, retrocede rápidamente. El coste de exploración es lineal respecto a N , pero la inicialización de la matriz sigue siendo cuadrática:

$$T_m(N, M) = C_{ini} + \sum_{i=1}^N (c_{gen} \cdot i + c_{crit}) \approx c_{14}N^2 + c'N^2$$

Por tanto, el orden del mejor caso queda limitado por la preparación de los datos:

$$T_m(N, M) \in \Theta(N^2)$$

3.6. Análisis experimental

Para el análisis experimental, hemos diseñado un programa en C++ muy similar al usado en el problema de avance rápido que evalúa el rendimiento del algoritmo en el mejor y el peor caso. En ambos escenarios mantenemos $M = 5$ como valor constante para poder obviar un parámetro de la función de tiempo del algoritmo, por las razones ya previamente mencionadas en la sección homónima del problema de avance rápido.

Cada caso de prueba para un valor de N determinado se ejecuta 10 veces y se calcula la mediana de los tiempos obtenidos (exceptuando los valores más altos del peor caso donde, debido a su alto coste, se realizan 3 ejecuciones).

Obviaremos la implementación del programa `tiempos.cpp`, pues es muy similar al del otro problema. De todos modos, el código puede encontrarse en el repositorio de GitHub.

Al ejecutar el programa, los tiempos medianos obtenidos (convertidos a milisegundos para facilitar su lectura) fueron los siguientes:

Cuadro 2: Resultados experimentales para Backtracking ($M = 5$).

Peor Caso		Mejor Caso		Mejor Caso (cont.)		Mejor Caso (cont.)	
N	T_M (ms)	N	T_m (ms)	N	T_m (ms)	N	T_m (ms)
10	0.390	100	0.048	1100	5.721	2100	21.999
15	4.199	200	0.189	1200	6.834	2200	23.850
20	11.092	300	0.423	1300	8.121	2300	26.550
25	27.161	400	0.758	1400	9.595	2400	28.613
30	73.879	500	1.212	1500	10.778	2500	31.063
35	174.563	600	1.713	1600	12.283		
40	260.881	700	2.322	1700	13.962		
45	504.439	800	3.020	1800	15.826		
50	917.586	900	3.817	1900	17.500		
		1000	4.716	2000	19.565		

Para visualizar la magnitud del impacto de las podas y la diferencia abismal entre ambos escenarios, se presentan a continuación las gráficas comparativas. En la escala lineal es evidente cómo el peor caso crece brutalmente ante pequeños incrementos de N , mientras que el mejor caso mantiene una progresión mucho más suave.

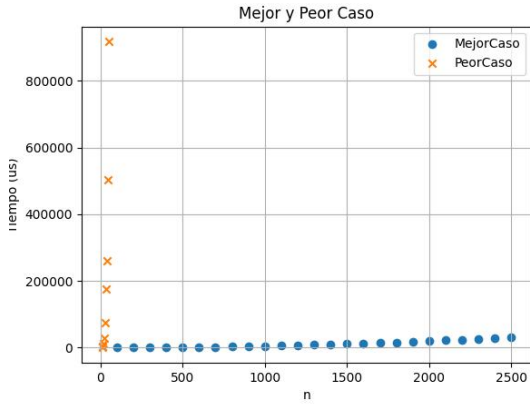


Figura 2: Comparativa Mejor vs Peor caso (Escala Lineal).

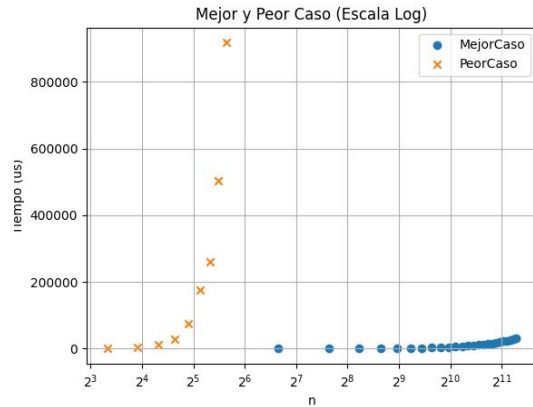


Figura 3: Comparativa en escala logarítmica.

El uso de la escala logarítmica permite apreciar que, aunque el mejor caso gestiona conjuntos de datos hasta 50 veces más grandes ($N = 2500$), el tiempo requerido sigue siendo órdenes de magnitud inferior al del peor caso con apenas $N = 50$.

3.7. Contraste

Finalmente, para contrastar el análisis teórico con el experimental, hemos utilizado un script en Python muy similar al usado en avance rápido. De nuevo, obviaremos su implementación, pero está disponible en el repositorio de GitHub del proyecto.

Debido a la naturaleza del problema, hemos analizado por separado el mejor y el peor caso, ya que, como hemos mencionado en el análisis teórico, la complejidad algorítmica es distinta para cada uno. Al igual que en el algoritmo de avance rápido, hemos mantenido M constante

para simplificar la representación gráfica.

Al ejecutar el script, obtenemos la siguiente salida por terminal, que confirma nuestras hipótesis teóricas con una precisión superior al 99.9 %:

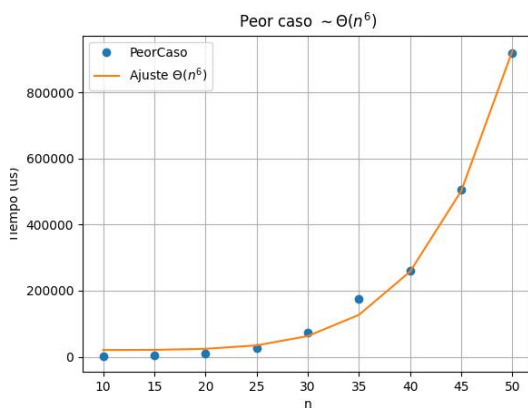
```
$ python3 regresion_bt.py

--- Análisis: Peor Caso ---
Modelo:  $O(n^6)$ 
R2: 0.999161

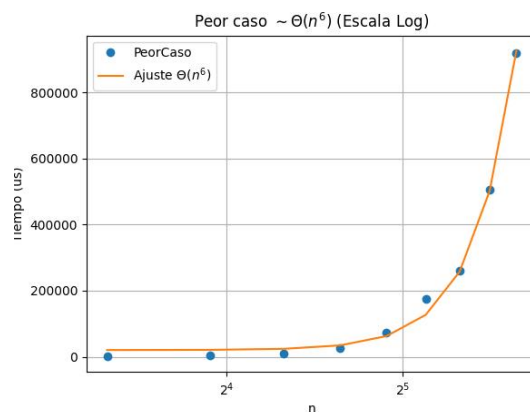
--- Análisis: Mejor Caso ---
Modelo:  $O(n^2)$ 
R2: 0.999695

Gráficas generadas con éxito en la carpeta assets/
```

Como podemos observar en las gráficas de contraste, los puntos experimentales se sitúan casi exactamente sobre la curva teórica calculada. En el peor caso, el ajuste al modelo $O(N^6)$ es evidente tanto en la escala lineal como en la logarítmica.



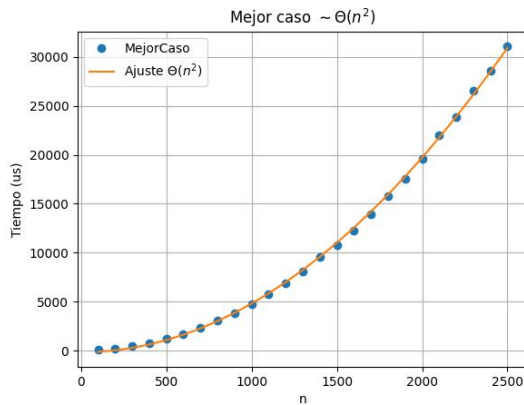
(a) Ajuste Peor Caso (Lineal)



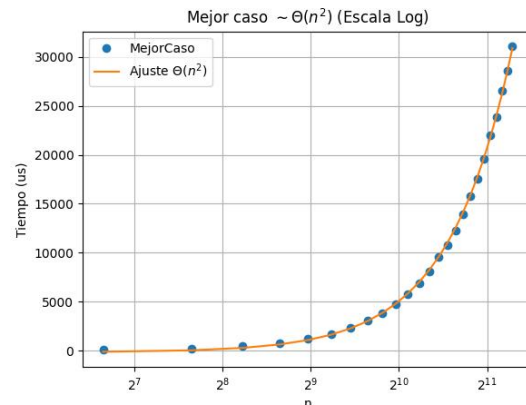
(b) Ajuste Peor Caso (Logarítmica)

Figura 4: Contraste del modelo teórico $O(N^6)$ en el peor escenario.

Del mismo modo, para el mejor caso, el contraste confirma que el tiempo de ejecución sigue una tendencia cuadrática $O(N^2)$, comportamiento derivado del coste de inicialización.



(a) Ajuste Mejor Caso (Lineal)



(b) Ajuste Mejor Caso (Logarítmica)

Figura 5: Contraste del modelo teórico $O(N^2)$ en el mejor escenario.

3.8. Estructura, compilación y ejecución

3.8.1. Estructura de ficheros

El proyecto de Backtracking se compone de los siguientes archivos principales localizados en el directorio `src/bt/`:

- **bt.cpp**: Implementación de la función principal del algoritmo.
- **bt_aux.hpp / bt_aux.cpp**: Definición de la estructura de datos `Solucion` y las funciones auxiliares (`Generar`, `Retroceder`, `Criterio`).
- **main.cpp**: Módulo de entrada que lee los datos del problema y ejecuta el algoritmo.
- **tiempos.cpp**: Programa diseñado para generar los escenarios de mejor y peor caso y realizar las mediciones temporales.
- **testsUnitarios.cpp**: Batería de pruebas automatizadas para verificar la corrección del algoritmo.
- **regresion_bt.py**: Script de análisis estadístico que genera las gráficas de complejidad.

3.8.2. Requisitos y dependencias

Para la compilación se requiere `g++` con estándar C++11 y la utilidad `make`. El análisis estadístico requiere Python 3.8+ con las bibliotecas estándar de ciencia de datos: `pandas`, `numpy`, `matplotlib` y `scipy`.

3.8.3. Instrucciones de uso

El proceso de compilación y ejecución se automatiza mediante el `Makefile` incluido:

```
$ make diversidad      # Compila el programa principal
$ make testsUnitarios  # Compila el ejecutable de pruebas
$ make tiempos         # Compila el medidor de rendimiento experimental
$ make                 # Compila todos los modulos del proyecto
```

```
Ejecutar las validaciones
$ ./testsUnitarios

Generar los datos experimentales (resultados_peor.csv y resultados_mejor.csv)
$ ./tiempos

Realizar el contraste estadístico y generar graficas
$ python3 regresion_bt.py

Eliminar binarios y archivos temporales
$ make clean
```

Los resultados del análisis gráfico se almacenan de forma automática en el subdirectorio `assets/`, facilitando el contraste visual de los modelos de complejidad.

4. Conclusión

A lo largo de este proyecto, además de aprender y reforzar nuestros conocimientos de programación en C++, hemos podido descubrir cómo se implementan y aplican las diversas técnicas de manejo de algoritmos que hemos estudiado a lo largo del cuatrimestre.

A parte de lo ya mencionado, este trabajo nos ha permitido mejorar nuestras capacidades de **trabajo en equipo** –debido a la necesidad de coordinación fuera de clase para avanzar, así como el reparto de trabajo de forma equitativa–, los conocimientos de **GitHub**, plataforma utilizada para compartir los ficheros editados entre nosotros, con el fin extra de facilitar el seguimiento del trabajo; y la **resolución de problemas** que requerían de cambiar nuestra forma de pensar –como, por ejemplo, el *ejercicio de avance rápido*, cuyo planteamiento tuvimos que cambiar en varias ocasiones hasta encontrar el que funcionaba correctamente.

Por último, cabe nombrar el uso de la *Inteligencia Artificial (IA)* en este trabajo. A lo largo de este proyecto, hemos utilizado herramientas de IA tal como *Gemini*, *Claude* o *Copilot* con fines tales como:

- Generación de códigos de ejemplo.
- Aclaración de dudas del propio enunciado del problema.
- Corrección de errores de nuestro propio código.
- Apoyo en la traducción de ciertas funciones de Pseudocódigo a C++.